Nutrient-Aware Prediction of Pathogen Suppression

Final Report on the MiCRM Dataset and Tabular Models

Pratham Shah
Ratzke Lab

March 2026-Present

Contents

1 The MiCRM Model and the Crucial Change **3**
1.1 The MiCRM equations . . . 3
1.2 The single-consumer, single-resource case and the R^* rule . . . 3
1.3 Mechanistic definition of suppression . . . 4
1.4 The crucial change: why the environment had to be held constant . . . 4
1.5 The final dataset . . . 5
1.6 Figures describing the dataset and the simulation . . . 5

2 Errors in the Previous Simulations **8**

3 Structure of the Model Scripts, and Group-Aware Splitting **8**
3.1 Shared pipeline . . . 8
3.2 The three feature versions . . . 9
3.3 GroupKFold . . . 9
3.4 GroupShuffleSplit . . . 9
3.5 How the two differ, and what the numbers mean . . . 10

4 Data Leakage Fixes **10**

5 Detailed Model Results **11**
5.1 Linear baselines . . . 11
5.2 Interpolation reference (row-wise split) . . . 11
5.3 Per-condition breakdown . . . 12
5.3.1 By diversity level . . . 12
5.3.2 By community size . . . 16

6 Held-Out-Broth Generalisation **19**

6.1 Reading the Pearson and Spearman numbers 20

7 Attribution: SHAP and Feature-Importance Patterns **20**

8 Further Steps: Optimisation for Nutrient Prescription **22**

9 References **22**

1 The MiCRM Model and the Crucial Change

This section restates the Microbial Consumer-Resource Model (MiCRM) equations, works out the single-consumer, single-resource case in full, and uses that case to explain the single most important change made for the final dataset.

1.1 The MiCRM equations

The model tracks two kinds of quantity at once: the abundance $N_i(t)$ of each species i, and the concentration $R_\alpha(t)$ of each carbon resource α. With S species and M resources this is a coupled system: the state at any instant is a single vector

$$y(t) = \big(N_1, \ldots, N_S,\ R_1, \ldots, R_M\big),$$

and the whole system is one point moving through an $(S + M)$-dimensional space. The dynamics are written as an ordinary differential equation $\mathrm{d}y/\mathrm{d}t = f(t, y)$, where f returns the instantaneous slope of every component."Coupled" means that the slope of each species depends on the resource levels, and the slope of each resource depends on the species abundances, so nothing evolves in isolation.

The two governing equations, exactly as implemented in the generator, are

$$\frac{\mathrm{d}N_i}{\mathrm{d}t} = N_i\, g_i \left(\sum_\alpha c_{i\alpha}\, w_\alpha (1 - l_\alpha)\, R_\alpha - m_i \right) \;-\; \delta\, N_i, \tag{1}$$

$$\frac{\mathrm{d}R_\alpha}{\mathrm{d}t} = \delta\,(R_{0,\alpha} - R_\alpha) \;-\; \sum_i c_{i\alpha}\, R_\alpha N_i \;+\; \sum_i \sum_\beta D_{i,\alpha\beta}\, l_\beta\, c_{i\beta}\, R_\beta N_i. \tag{2}$$

The symbols are: $c_{i\alpha}$ the uptake rate of species i on resource α; w_α the energy value (quality) of resource α; l_α the fraction of resource α that is leaked back to the environment rather than used; m_i the maintenance cost; g_i a growth conversion factor; δ the dilution (chemostat) rate; $R_{0,\alpha}$ the supplied concentration of resource α (this is the *broth*); and $D_{i,\alpha\beta}$ the cross-feeding matrix that redistributes leaked β into secreted α.

In words: equation (1) says a species grows in proportion to the net energy it harvests (intake minus maintenance) and is washed out at rate δ; equation (2) says each resource is supplied by dilution toward $R_{0,\alpha}$, drawn down by consumption, and partly replenished by cross-feeding.

1.2 The single-consumer, single-resource case and the R^* rule

To see the mechanism cleanly, reduce to one species, one resource, and no cross-feeding ($D = 0$). Equations (1)–(2) collapse to

$$\frac{\mathrm{d}N}{\mathrm{d}t} = N\Big[\, g\big(c\, w\, (1 - l)\, R - m\big) - \delta \,\Big], \tag{3}$$

$$\frac{\mathrm{d}R}{\mathrm{d}t} = \delta\,(R_0 - R) - c\, R\, N. \tag{4}$$

A **steady state** (fixed point) is a state where nothing changes, $\mathrm{d}N/\mathrm{d}t = 0$ and $\mathrm{d}R/\mathrm{d}t = 0$. For a surviving species ($N^* > 0$) the bracket in the first equation must vanish:

$$g\big(c\, w\, (1 - l)\, R^* - m\big) - \delta = 0 \qquad \Longrightarrow \qquad \boxed{R^* = \frac{m + \delta/g}{c\, w\, (1 - l)}}.$$

The quantity R^* is the resource concentration at which the species exactly breaks even: above R^* it grows, below R^* it shrinks. The essential point is that R^* depends only on the species' own traits (m, c, w) and on the environment parameters (δ, l, w). R_0 sets where the resource starts; R^* sets where any single competitor drives it.

Now extend to two consumers competing for one resource. Both draw the resource down. The species with the *lower* R^* can still grow at resource levels where the other is already shrinking, so it keeps consuming, pushes R down to its own R^*, and starves the higher-R^* competitor out. This is ***Tilman's R^* rule***: ***competitive exclusion happens through resource drawdown***, and the winner is simply whoever has the lowest R^*.

1.3 Mechanistic definition of suppression

This gives a clean mechanistic reading of what "suppression" means in the model. The pathogen P has its own break-even level R_P^* on each resource it eats. The community suppresses the pathogen precisely when it contains members whose R^* on the pathogen's resources is lower than R_P^*. Those members draw the shared resources below the level the pathogen needs to break even, so the pathogen declines. Suppression is therefore not about "how many" species are present in the abstract; it is about whether the supplied nutrients (R_0) feed community members that out-compete the pathogen for exactly those nutrients.

In the dataset, suppression is measured as

$$\text{suppression} = 1 - \frac{N_P^{(\text{in community})}}{N_P^{(\text{alone})}}, \qquad \text{clipped to } [0, 1],$$

comparing the pathogen's abundance in the community against its abundance grown alone in the same broth.

1.4 The crucial change: why the environment had to be held constant

As mentioned above, the R^* depends on parameters such as w, l, c, etc. In the earlier datasets, the environment parameters w, l, and δ were drawn *randomly and independently for every broth.* That means the same supplied nutrient vector R_0 produced a *different* R^* in every broth, because w was different each time. Two broths could have identical R_0 and yet give opposite competitive outcomes and hence opposite suppression, purely because of their hidden, random parameters.

This is fatal for a model that only sees R_0. The map from nutrients to suppression was scrambled by a variable (w, l, δ) that was never given to the model as a feature. No amount of data or model capacity can recover a mapping that the data-generating process itself made non-functional. This is the true reason the earlier held-out-broth tests collapsed (Section 2), and it is captured in the generator as `FIX 2`:

> **FIX 2 (`CONSTANT_ENV = True`).** Draw w, l, δ *once* and share them across every broth. Now, the nutrient-to-suppression relationship becomes a genuine deterministic function that a model can learn and transfer.

A second, supporting change addresses a different problem. Even with a clean mapping, the previous versions had only 20-32 distinct broths , so R_0 took only 20-32 distinct values. This is `FIX 1`:

FIX 1 (many broths). Generate 30 random broths per diversity level, giving 150 distinct broths. Active nutrient sets and concentrations are re-sampled for each, so R_0 now densely samples the nutrient space. Held-out broths fall near training broths, and generalisation becomes interpolation in nutrient space rather than extrapolation to an unseen category.

Together, `FIX 2` makes the mapping learnable and `FIX 1` makes it testable.

1.5 The final dataset

The generator simulates 101 species (index 0 is the pathogen; the remaining 100 are the community pool, split into 50 specialists and 50 generalists) over 20 carbon resources. The design is:

- **Broths.** 150 total = 30 per diversity level, with diversity level $\in \{1, 2, 4, 8, 16\}$ (the number of nutrients actually supplied). Each broth picks a random active nutrient set of that size, with concentrations that sum to a fixed total carbon of 20.
- **Constant environment.** w, l, δ are shared across all broths (`FIX 2`); the CSV confirms a single dilution value ($\delta = 0.0993$) and a single R_0 total for every row.
- **Communities.** For each broth and each community size $\in \{12, 16, 24, 32, 48\}$, 100 random communities are drawn from the 100-strain pool. This gives $150 \times 5 \times 100 = 75{,}000$ communities.
- **Target.** `community_suppression`, defined above; across the dataset it is well spread (mean 0.517, standard deviation 0.333).

1.6 Figures describing the dataset and the simulation

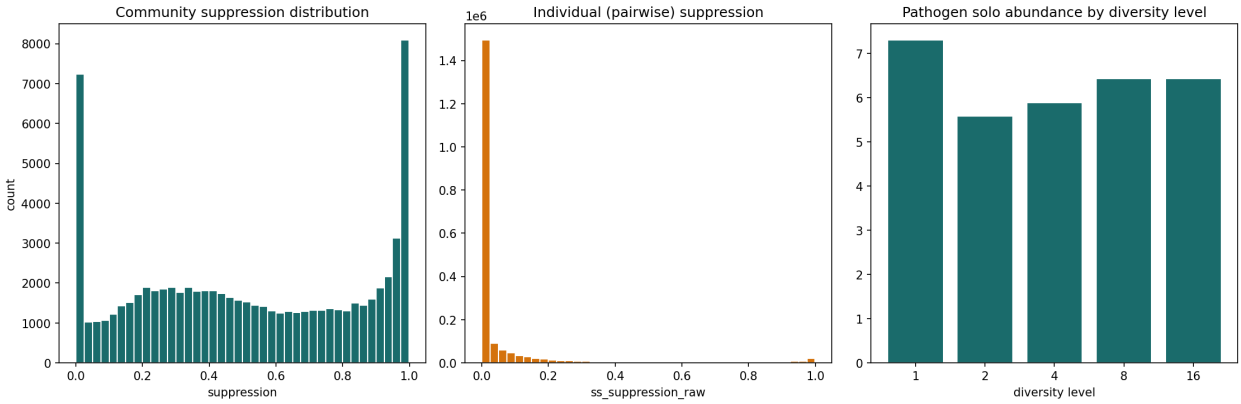


Figure 1: Suppression value and pathogen abundance distribution

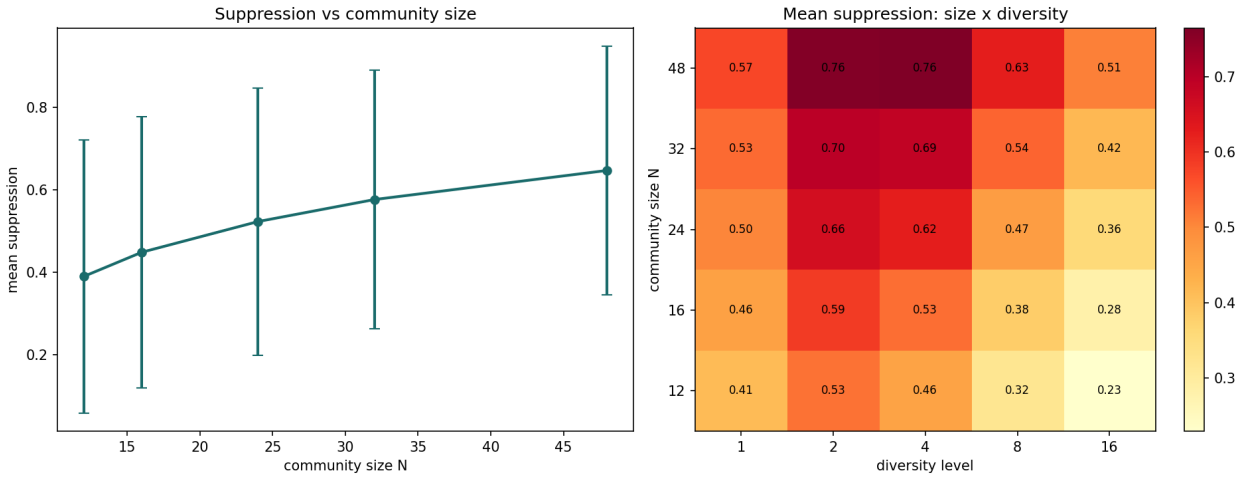


Figure 2: Suppression values by community size and diversity levels

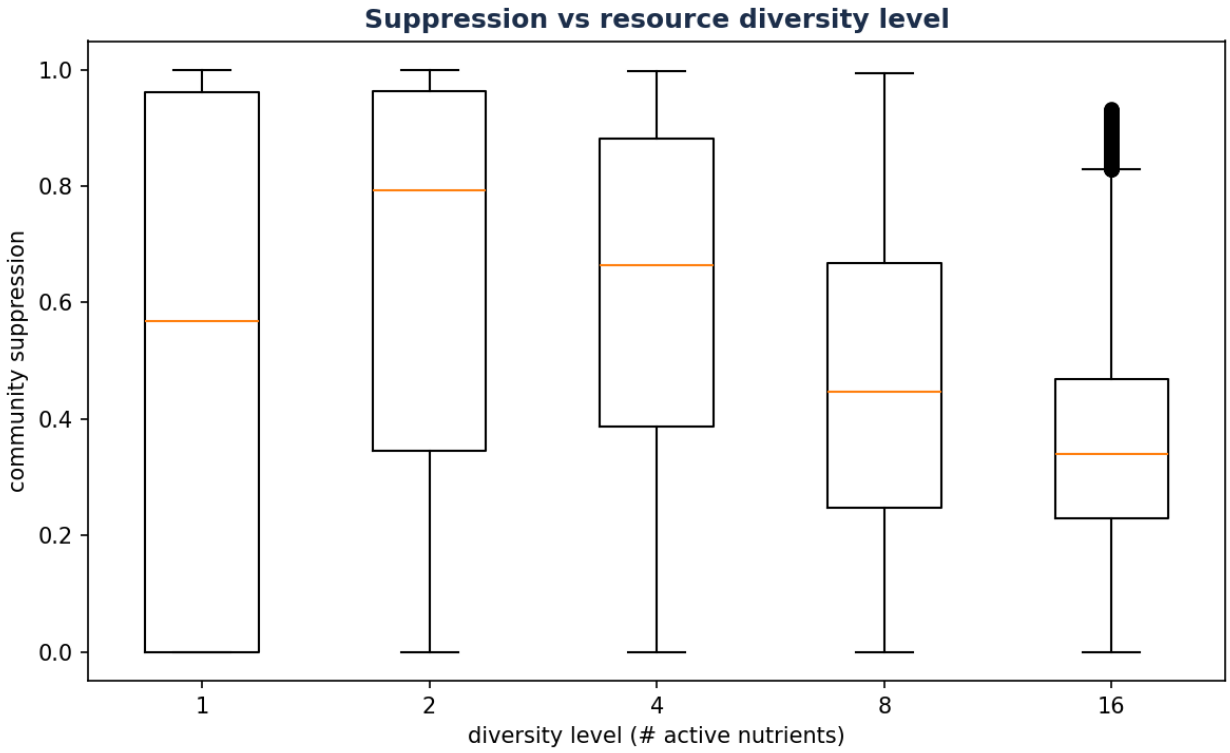


Figure 3: Suppression values by diversity levels

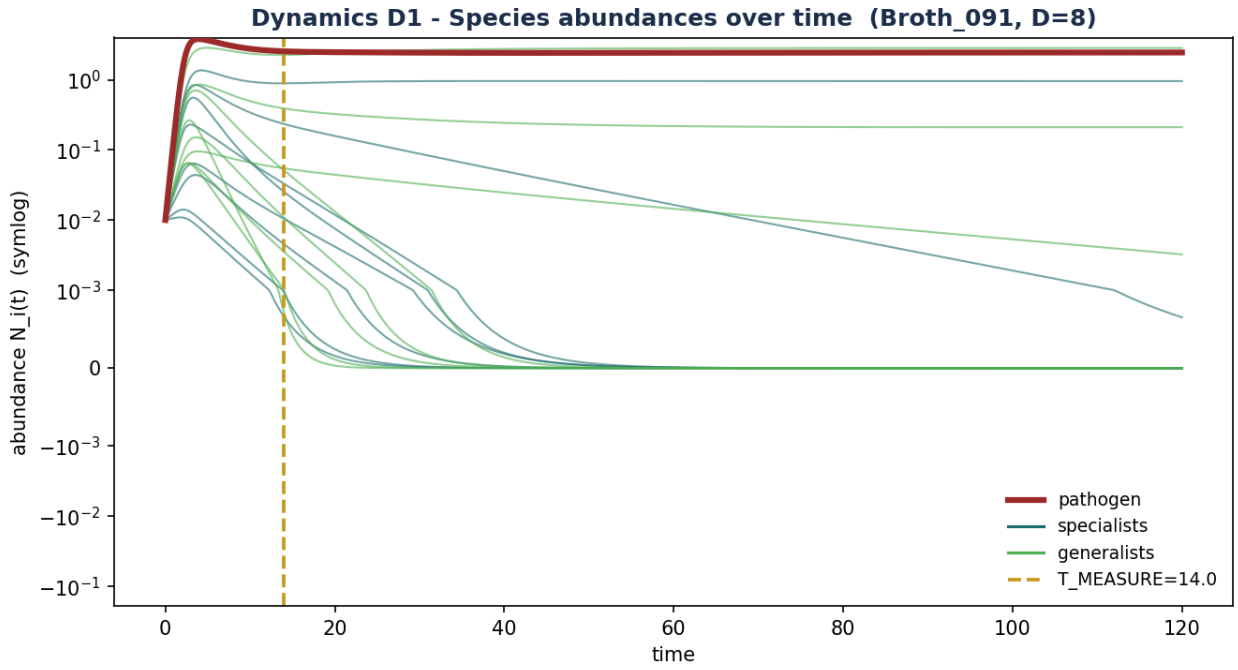


Figure 4: Species abundance dynamics of a random broth, for example. The yellow dotted line shows the time point at which we took our measurement. Clearly, we can see that the measurement is taken at a time when the dynamics begin to steady out

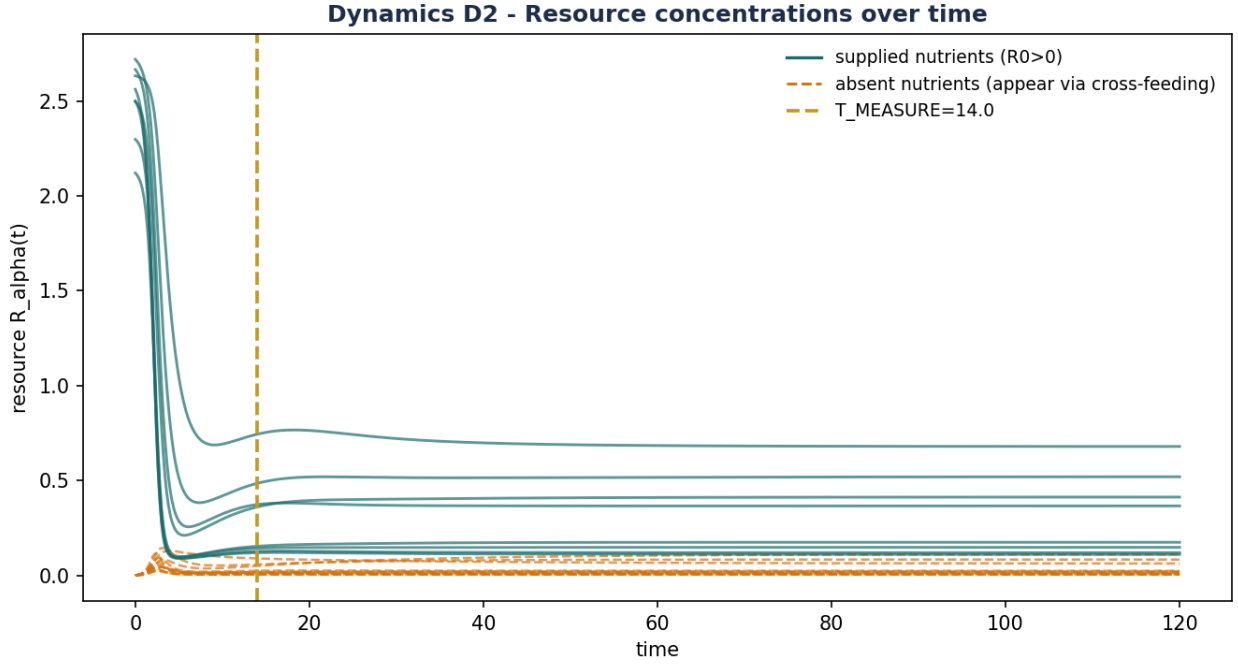


Figure 5: Resource concentration dynamics of a random broth, for example. The yellow dotted line shows the time point at which we took our measurement. Clearly, we can see that the measurement is taken at a time when the dynamics begin to steady out

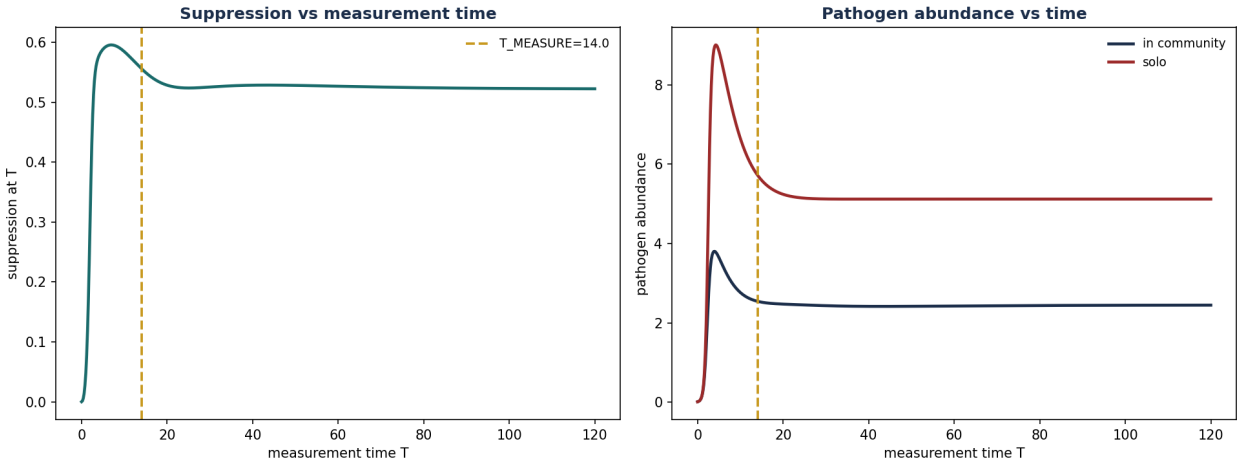


Figure 6: Justification for using the T measure=14

2 Errors in the Previous Simulations

Four problems in the previous runs motivated the changes above and the leakage fixes to be discussed in Section 4.

1. Data leakage in evaluation. The final model passed the test set as an early-stopping validation set, the hyperparameter search used a shuffled `KFold` instead of a group-aware split, and the held-out-broth loop used the held-out fold itself for early stopping. All four let information leak from the evaluation data into model selection, inflating the reported scores. These are fixed in detail in Section 4.

2. Too few nutrient environments. Mentioned above.

3. Varying environments without passing required variables. Mentioned above.

4. Misleading metric aggregation. Averaging per-fold R^2 over small, distribution-shifted folds produced unstable and misleading numbers (for example -0.15 ± 1.01). The corrected pipeline reports rank and linear correlations (Pearson, Spearman) alongside R^2 and uses larger, balanced folds (150 broths, 30 held out per fold), which is what makes the generalisation signal in Section 6 readable.

3 Structure of the Model Scripts, and Group-Aware Splitting

3.1 Shared pipeline

All four model families (XGBoost, LightGBM, CatBoost, and the linear baselines) share one structure; only the syntax specific to each model changes. The stages are:

1. **Load and assemble features.** Read `community_index_v10.csv` and build the feature matrix from four blocks: 100 species-membership columns, 20 R_0 concentrations, 20 active flags, and 2 structural columns (`community_size`, `diversity_level`).
2. **Row split and group labels.** A stratified row-wise `train_test_split` (80/20) provides an interpolation reference. Separately, the broth label of each training row is extracted

(`groups_train`) so that later steps can respect broth boundaries.

3. **Baselines.** A global-mean predictor and three linear models (Ridge, Lasso, Elastic Net), fitted with cross-validated regularisation (`RidgeCV`, `LassoCV`, `ElasticNetCV`), which each try a grid of penalty strengths and keep the value with the best cross-validated error.
4. **Model fit with an inner holdout.** The tree model is fitted with early stopping, where the early-stopping set is an *inner* holdout carved only from training broths (the `inner_holdout` helper), never the test set.
5. **Tuning.** `GridSearchCV` over a small parameter grid, using `GroupKFold(3)` so tuning folds never share a broth, and a base estimator with no early stopping so the search cannot peek.
6. **Interpolation reference.** The tuned model is retrained and scored on the held-out 20% rows.
7. **Held-out-broth evaluation.** `GroupKFold(5)` and `GroupShuffleSplit(20)` loops, each reporting R^2, MAE, RMSE, Pearson, and Spearman (Section 6).
8. **Per-condition breakdown.** Metrics recomputed within each diversity level and each community size (Section 5.3).
9. **Attribution and figures.** Built-in importance, SHAP values, and a six-panel figure.

3.2 The three feature versions

Every script is run in three versions that differ only in the feature set:

Version	Features	Count
V1 Full	species + R_0 + active + structural	142
V2 Nutrients + size	R_0 + active + structural	42
V3 Pure nutrients	R_0 + active + diversity level	41

3.3 GroupKFold

Ordinary k-fold cross-validation shuffles individual rows into folds. That is wrong here because **many rows share a broth**: a random split places communities from the same broth into both train and test, so the model is scored on nutrient environments it has already seen. `GroupKFold` fixes this by partitioning the *groups* (broths), not the rows, into k folds. Every row from a given broth is placed entirely in one fold. With 150 broths and $k = 5$, each fold holds out 30 whole broths (15,000 rows); each broth is tested exactly once across the five folds. The guarantee is that the train and test broth sets never intersect, ***so each fold score measures prediction on genuinely unseen nutrient environments***.

3.4 GroupShuffleSplit

`GroupShuffleSplit` is the randomised counterpart. Instead of an exhaustive partition, it repeatedly draws a random subset of broths (here 20% per split) as the test set, keeping broths intact, and repeats this many times (here 20 splits). A broth may be tested several times or not at all. It does not test each broth exactly once, but it measures how sensitive the held-out performance is to which broths happen to be held out, which is a useful robustness check.

3.5 How the two differ, and what the numbers mean

`GroupKFold` gives a clean, deterministic estimate: every broth is tested once, so the mean over folds is a balanced held-out-broth score. `GroupShuffleSplit` gives a distribution over random broth choices; if its mean agrees with the `GroupKFold` mean, the generalisation estimate is not an artifact of one particular fold assignment. In practice the two agree closely in v10 (compare Tables 10 and 11), which is itself evidence that the ***held-out-broth performance is stable***.

4 Data Leakage Fixes

The corrected code removes three specific leaks.

Leak 1: the test set as an early-stopping set. The original final model was fitted with `eval_set = [(X_train, y_train), (X_test, y_test)]` and early stopping on, so the test set chose the number of boosting rounds. The test samples influenced model selection even though they were not used for gradient updates, so the test set was no longer independent. Fix: split the training data into a fit set and an inner validation set, both drawn only from training broths, and use `eval_set = [(X_fit, y_fit), (X_val, y_val)]`. The test set is touched once, after fitting, for prediction only.

Leak 2: shuffled tuning folds. The hyperparameter search used a shuffled `KFold`, which mixes broths across CV folds and tunes to interpolation performance. Fix: `GroupKFold` inside `GridSearchCV`, passing the broth labels as `groups=groups_train`, so no broth spans two tuning folds.

Leak 3: the held-out fold as its own validation set. In the group generalisation loop the held-out broths were reused as the early-stopping set, `m.fit(X_tr, y_tr, eval_set=[(X_te, y_te)])`, so early stopping selected the iteration using the very broths being scored. Fix: add an inner validation split *inside* each outer training fold, drawn only from the outer training broths, and use it for early stopping. The outer held-out broths are used only for the final prediction.

Finally, explicit overlap checks assert $\text{set}(\boldsymbol{groups}_{train}) \cap \text{set}(\boldsymbol{groups}_{test}) = \varnothing$ ***for every group split, so no broth can silently appear on both sides.***

5 Detailed Model Results

The models tested include: Linear Baselines (Ridge, Lasso, and Elastic Net Regression), XGBoost, LightGBM, CatBoost, TabPFN-3.

5.1 Linear baselines

Ridge, Lasso, and Elastic Net are computed once (in the XGBoost scripts) and shared as the linear reference for all models. Their alphas are chosen by cross-validation; Lasso and Elastic Net additionally report how many features they keep.

Table 1: Linear baselines. Ridge, Lasso, and Elastic Net agree to within rounding in every version, and Lasso keeps almost all features, so the linear signal is dense rather than sparse.

Version	Feat.	Ridge R^2	Lasso R^2	ElasticNet R^2	MAE	Lasso kept
V1 Full	142	0.6425	0.6426	0.6426	0.1536	134/142
V2 Nutrients+size	42	0.6174	0.6174	0.6174	0.1604	–
V3 Pure nutrients	41	0.5470	0.5470	0.5470	0.1777	–

5.2 Interpolation reference (row-wise split)

Table 2: Tuned models on the held-out 20% rows (interpolation reference). Initial (pre-tuning) scores were within a few thousandths; tuning helped only marginally.

Model	Version	R^2	MAE	RMSE
XGBoost	V1	0.8910	0.0618	0.1098
XGBoost	V2	0.7482	0.1122	0.1668
XGBoost	V3	0.6620	0.1398	0.1932
LightGBM	V1	0.8986	0.0578	0.1059
LightGBM	V2	0.7484	0.1125	0.1667
LightGBM	V3	0.6621	0.1408	0.1932
CatBoost	V1	0.9089	0.0544	0.1003
CatBoost	V2	0.7582	0.1108	0.1634
CatBoost	V3	0.6712	0.1400	0.1906
TabPFN	V1	0.8909	0.0642	0.1098
TabPFN	V2	0.7957	0.0987	0.1502
TabPFN	V3	0.6943	0.1283	0.1838

Observations

- Two things are noticeable first. The tested models perform significantly better than the linear baselines for all the 3 versions. However, the performance gap becomes lesser as we keep on reducing the features, which shows that our regression problem is approaching a linear mapping.

- XGBoost and LightGBM have an almost identical performance across all the 3 versions.
- For V1, CatBoost leads in terms of the highest R^2 and the lowest error values.
- However, as we keep reducing features (V2 and V3), we can clearly see that TabPFN performs better on both those versions than of the tree models, both in terms of higher R^2 and lower error values.

5.3 Per-condition breakdown

Another informative diagnostic is how the metrics behave within each diversity level and each community size.

5.3.1 By diversity level

The diversity pattern is not consistent across versions in R^2, but it is consistent in MAE values.

The resolution is that R^2 is not exactly the most important metric we should look at here: std(y) collapses from 0.43 at D=1 to 0.17 at D=16, and R^2 divides by it. At high diversity there is simply very little variance to explain, so R^2 looks bad even when the absolute error is tiny. In V1 the models are accurate enough (small MSE) to overcome the shrinking denominator and R^2 still recovers; in V2 and V3 they are less accurate, so the same denominator collapse drags R^2 down instead of up.

The MAE pattern however is consistent: ***in every version and every model, MAE rises from D=1 to a peak at D=4 and then falls monotonically through D=8 to D=16***. There are no exceptions across the twelve rows.

Table 3: Full model (V1) by diversity level, all four models.

Model	D	R^2	MAE	RMSE	Pearson	Spearman	Bias
XGBoost	1	0.9722	0.0396	0.0722	0.9861	0.9628	−0.0010
	2	0.7909	0.0903	0.1630	0.8917	0.8997	0.0196
	4	0.7492	0.0994	0.1474	0.8691	0.8607	−0.0172
	8	0.9318	0.0486	0.0679	0.9657	0.9639	−0.0005
	16	0.9287	0.0314	0.0460	0.9660	0.9710	−0.0046
LightGBM	1	0.9751	0.0388	0.0684	0.9876	0.9618	−0.0004
	2	0.7896	0.0854	0.1635	0.8901	0.9007	0.0167
	4	0.7822	0.0922	0.1374	0.8875	0.8859	−0.0111
	8	0.9409	0.0443	0.0632	0.9702	0.9688	0.0003
	16	0.9409	0.0284	0.0419	0.9718	0.9744	−0.0056
CatBoost	1	0.9781	0.0333	0.0641	0.9891	0.9656	−0.0010
	2	0.8104	0.0824	0.1552	0.9017	0.9131	0.0147
	4	0.8103	0.0860	0.1282	0.9036	0.9024	−0.0097
	8	0.9388	0.0436	0.0643	0.9692	0.9661	0.0033
	16	0.9480	0.0267	0.0393	0.9754	0.9788	−0.0031
TabPFN	1	0.8996	0.0649	0.1372	0.9498	0.9583	−0.0173
	2	0.8648	0.0797	0.1310	0.9301	0.9284	0.0005
	4	0.8104	0.0879	0.1282	0.9007	0.9110	−0.0084
	8	0.9084	0.0577	0.0787	0.9537	0.9496	−0.0083
	16	0.9446	0.0306	0.0405	0.9729	0.9705	−0.0049

The mechanism behind the D=4 peak is the same across versions. At D=1 a single nutrient is supplied, so suppression is a clean single-resource competition and is easy to predict. At high diversity many nutrients are present at low individual concentration, outcomes saturate toward strong suppression (some competitor almost always wins), and the target becomes nearly deterministic again. The middle band, and ***D=4 specifically, is where the number of nutrients is large enough to make the multi-resource R^* competition more complex, but not yet large enough for averaging to wash the outcome out***. That is the hardest regime, and every model feels it at the same place. TabPFN is the flattest through this band (its D=2 and D=4 MAE rise the least above D=1), consistent with its general smoothness, but it too peaks at D=4.

Table 4: Nutrient + size model (V2) by diversity level, all four models.

Model	D	R^2	MAE	RMSE	Pearson	Spearman	Bias
XGBoost	1	0.8829	0.0792	0.1482	0.9396	0.9459	−0.0026
	2	0.6278	0.1439	0.2174	0.7960	0.8163	0.0247
	4	0.5168	0.1587	0.2046	0.7247	0.7158	−0.0268
	8	0.7273	0.1046	0.1358	0.8529	0.8376	−0.0005
	16	0.6784	0.0745	0.0977	0.8245	0.8161	−0.0047
LightGBM	1	0.8818	0.0796	0.1489	0.9391	0.9446	−0.0046
	2	0.6108	0.1473	0.2223	0.7831	0.8048	0.0168
	4	0.5573	0.1541	0.1959	0.7517	0.7580	−0.0203
	8	0.7230	0.1053	0.1368	0.8506	0.8376	−0.0044
	16	0.6532	0.0761	0.1014	0.8096	0.8118	−0.0066
CatBoost	1	0.8829	0.0810	0.1482	0.9397	0.9455	−0.0014
	2	0.6433	0.1409	0.2129	0.8056	0.8279	0.0240
	4	0.5583	0.1526	0.1956	0.7498	0.7610	−0.0107
	8	0.7240	0.1053	0.1366	0.8510	0.8365	0.0015
	16	0.6845	0.0741	0.0967	0.8283	0.8183	−0.0035
TabPFN	1	0.8614	0.0796	0.1612	0.9302	0.9456	−0.0203
	2	0.7655	0.1125	0.1726	0.8753	0.8753	0.0091
	4	0.6488	0.1271	0.1744	0.8060	0.8174	−0.0015
	8	0.7377	0.1017	0.1331	0.8592	0.8446	−0.0016
	16	0.6989	0.0725	0.0945	0.8366	0.8202	0.0006

Table 5: Pure-nutrient model (V3) by diversity level, all four models.

Model	D	R^2	MAE	RMSE	Pearson	Spearman	Bias
XGBoost	1	0.8488	0.0933	0.1684	0.9213	0.9375	0.0002
	2	0.5595	0.1657	0.2366	0.7517	0.7774	0.0239
	4	0.3917	0.1873	0.2296	0.6345	0.6220	−0.0266
	8	0.5516	0.1400	0.1741	0.7428	0.7101	−0.0013
	16	0.3470	0.1127	0.1392	0.5907	0.5608	−0.0040
LightGBM	1	0.8490	0.0940	0.1683	0.9214	0.9371	−0.0023
	2	0.5878	0.1641	0.2288	0.7688	0.7928	0.0144
	4	0.3525	0.1925	0.2369	0.6024	0.5749	−0.0288
	8	0.5515	0.1403	0.1741	0.7429	0.7129	−0.0034
	16	0.3419	0.1131	0.1397	0.5872	0.5606	−0.0056
CatBoost	1	0.8489	0.1008	0.1683	0.9217	0.9375	0.0043
	2	0.5741	0.1655	0.2326	0.7606	0.7889	0.0187
	4	0.4309	0.1796	0.2221	0.6617	0.6706	−0.0181
	8	0.5510	0.1407	0.1742	0.7428	0.7104	0.0003
	16	0.3419	0.1132	0.1397	0.5912	0.5595	−0.0055
TabPFN	1	0.8023	0.1009	0.1926	0.9010	0.9375	−0.0318
	2	0.6883	0.1340	0.1990	0.8306	0.8323	0.0116
	4	0.4946	0.1573	0.2093	0.7042	0.7150	0.0067
	8	0.5608	0.1384	0.1723	0.7491	0.7177	0.0032
	16	0.3681	0.1109	0.1369	0.6078	0.5665	0.0050

5.3.2 By community size

Table 6: Full model (V1) by community size, all four models. std(y) = 0.327, 0.331, 0.323, 0.312, 0.304 for N = 12, 16, 24, 32, 48.

Model	N	R^2	MAE	RMSE	Pearson	Spearman	Bias
XGBoost	12	0.8467	0.0807	0.1282	0.9209	0.9038	−0.0027
	16	0.8656	0.0753	0.1213	0.9310	0.9270	−0.0029
	24	0.8994	0.0589	0.1025	0.9489	0.9498	−0.0014
	32	0.8997	0.0524	0.0988	0.9488	0.9520	0.0009
	48	0.9047	0.0418	0.0939	0.9512	0.9539	0.0024
LightGBM	12	0.8609	0.0749	0.1221	0.9286	0.9124	−0.0020
	16	0.8810	0.0685	0.1141	0.9391	0.9340	−0.0010
	24	0.9061	0.0551	0.0990	0.9522	0.9523	0.0005
	32	0.8981	0.0508	0.0996	0.9478	0.9514	0.0007
	48	0.9095	0.0396	0.0915	0.9538	0.9592	0.0016
CatBoost	12	0.8656	0.0747	0.1200	0.9312	0.9132	−0.0021
	16	0.8823	0.0685	0.1135	0.9400	0.9334	0.0013
	24	0.9182	0.0512	0.0925	0.9589	0.9591	−0.0002
	32	0.9215	0.0436	0.0874	0.9602	0.9634	0.0012
	48	0.9261	0.0340	0.0826	0.9624	0.9668	0.0040
TabPFN	12	0.8086	0.0903	0.1433	0.8998	0.8864	−0.0108
	16	0.8396	0.0811	0.1325	0.9167	0.9127	−0.0086
	24	0.8931	0.0632	0.1057	0.9453	0.9440	−0.0070
	32	0.9219	0.0526	0.0872	0.9606	0.9593	−0.0092
	48	0.9632	0.0337	0.0584	0.9815	0.9789	−0.0027

The community-size pattern, and how it differs by version. For V1 and V2, it is consistent: ***MAE falls monotonically as communities grow, and both correlations rise, in every model*** (Table 9). A larger community is a larger random draw from the same 100-strain pool, so the chance it contains at least one strong low-R^* competitor for the supplied nutrients rises, and the pathogen is driven close to fully suppressed. The outcome becomes more deterministic and less sensitive to which particular strains were drawn, so every model predicts it more tightly. Small communities are the hard case: whether suppression happens can hinge on a single member.

V3 breaks the monotonicity a bit ***In the pure-nutrient model the MAE falls to a minimum at N=32 and then rises again at N=48 in all four models***. Without species information the model can only predict close to a per-broth average, so it over-predicts suppression for small communities (bias $\approx +0.13$ at N=12) and under-predicts for large ones (bias ≈ -0.13 at N=48); those large biases inflate the error at both ends and leave the middle (N=24–N=32) as the best-predicted band. The bias magnitudes are nearly identical across models ($+0.127 \to -0.130$ XGBoost, $+0.124 \to -0.134$ LightGBM, $+0.128 \to -0.129$ CatBoost, $+0.125 \to -0.129$ TabPFN), confirming it is a structural consequence of dropping the composition features rather than a property of any one learner. Note that the Spearman correlations still rise with size even in V3 ($0.76 \to 0.92$): the pure-nutrient model keeps *ranking* large communities correctly, it just mis-calibrates their absolute level.

Table 7: Nutrient + size model (V2) by community size, all four models.

Model	N	R^2	MAE	RMSE	Pearson	Spearman	Bias
XGBoost	12	0.6319	0.1416	0.1986	0.7950	0.7619	−0.0028
	16	0.6686	0.1345	0.1905	0.8177	0.8004	−0.0012
	24	0.7366	0.1154	0.1659	0.8585	0.8492	−0.0061
	32	0.7813	0.0975	0.1458	0.8840	0.8826	−0.0010
	48	0.8425	0.0719	0.1207	0.9180	0.9187	0.0012
LightGBM	12	0.6342	0.1419	0.1980	0.7965	0.7608	−0.0021
	16	0.6712	0.1345	0.1897	0.8193	0.8015	−0.0021
	24	0.7353	0.1153	0.1663	0.8579	0.8498	−0.0086
	32	0.7791	0.0982	0.1466	0.8828	0.8826	−0.0043
	48	0.8419	0.0726	0.1209	0.9177	0.9203	−0.0021
CatBoost	12	0.6408	0.1413	0.1962	0.8006	0.7619	0.0020
	16	0.6850	0.1318	0.1857	0.8278	0.8086	0.0031
	24	0.7472	0.1138	0.1625	0.8647	0.8548	−0.0018
	32	0.7895	0.0959	0.1431	0.8887	0.8884	0.0034
	48	0.8522	0.0710	0.1169	0.9234	0.9282	0.0032
TabPFN	12	0.6693	0.1307	0.1883	0.8190	0.7874	−0.0085
	16	0.7196	0.1191	0.1752	0.8491	0.8341	0.0009
	24	0.7802	0.1029	0.1515	0.8839	0.8753	−0.0051
	32	0.8356	0.0844	0.1265	0.9145	0.9100	−0.0010
	48	0.9161	0.0561	0.0881	0.9572	0.9522	0.0001

Across models, TabPFN is again the standout at the size extremes: in V1 it starts worst at N=12 (MAE 0.090) but finishes best at N=48 (R^2 = 0.963, the highest single-slice score in the whole breakdown), the steepest size-improvement of any model, and in V2 and V3 it carries the lowest MAE at essentially every size. CatBoost is second, with the largest V1 drop and the joint-lowest V1 N=48 error.

In summary: the diversity breakdown is consistent once read in MAE (hardest at D=4 everywhere), the size breakdown is consistent in the full-information versions (monotone improvement), CatBoost is the most robust tree wherever the problem is hardest, and TabPFN is the standout on reduced features and at large community sizes.

Table 8: Pure-nutrient model (V3) by community size, all four models. The Bias column is the story: every model over-predicts for small communities (positive bias at N=12) and under-predicts for large ones (negative bias at N=48).

Model	N	R^2	MAE	RMSE	Pearson	Spearman	Bias
XGBoost	12	0.4578	0.1748	0.2411	0.7801	0.7564	0.1267
	16	0.6252	0.1427	0.2026	0.8168	0.8039	0.0677
	24	0.7373	0.1173	0.1657	0.8600	0.8516	−0.0108
	32	0.7401	0.1159	0.1590	0.8828	0.8859	−0.0611
	48	0.6234	0.1484	0.1866	0.8985	0.9207	−0.1303
LightGBM	12	0.4654	0.1743	0.2394	0.7801	0.7577	0.1235
	16	0.6302	0.1427	0.2012	0.8168	0.8058	0.0636
	24	0.7347	0.1185	0.1664	0.8593	0.8523	−0.0147
	32	0.7381	0.1175	0.1596	0.8843	0.8885	−0.0643
	48	0.6144	0.1511	0.1888	0.8989	0.9222	−0.1337
CatBoost	12	0.4672	0.1751	0.2390	0.7870	0.7608	0.1277
	16	0.6368	0.1421	0.1994	0.8254	0.8106	0.0694
	24	0.7471	0.1170	0.1625	0.8664	0.8569	−0.0088
	32	0.7480	0.1168	0.1566	0.8871	0.8907	−0.0599
	48	0.6343	0.1488	0.1839	0.9024	0.9269	−0.1288
TabPFN	12	0.4814	0.1660	0.2358	0.7968	0.7806	0.1250
	16	0.6583	0.1318	0.1934	0.8395	0.8306	0.0695
	24	0.7724	0.1057	0.1542	0.8795	0.8698	−0.0094
	32	0.7780	0.1036	0.1470	0.9042	0.9008	−0.0616
	48	0.6699	0.1344	0.1747	0.9225	0.9393	−0.1287

Table 9: MAE by community size, every model and version. V1 and V2 fall monotonically; V3 reaches its minimum at N=32 and rises at N=48, the signature of the pure-nutrient bias sign-flip.

Model	Ver	N=12	N=16	N=24	N=32	N=48
XGBoost	V1	0.0807	0.0753	0.0589	0.0524	0.0418
	V2	0.1416	0.1345	0.1154	0.0975	0.0719
	V3	0.1748	0.1427	0.1173	0.1159	0.1484
LightGBM	V1	0.0749	0.0685	0.0551	0.0508	0.0396
	V2	0.1419	0.1345	0.1153	0.0982	0.0726
	V3	0.1743	0.1427	0.1185	0.1175	0.1511
CatBoost	V1	0.0747	0.0685	0.0512	0.0436	0.0340
	V2	0.1413	0.1318	0.1138	0.0959	0.0710
	V3	0.1751	0.1421	0.1170	0.1168	0.1488
TabPFN	V1	0.0903	0.0811	0.0632	0.0526	0.0337
	V2	0.1307	0.1191	0.1029	0.0844	0.0561
	V3	0.1660	0.1318	0.1057	0.1036	0.1344

6 Held-Out-Broth Generalisation

Table 10: GroupKFold (5 folds) over held-out broths, v10. Mean across folds. Standard deviations are roughly 0.10–0.18 on R^2 and 0.06–0.13 on the correlations.

Model	Version	R^2	MAE	RMSE	Pearson	Spearman
XGBoost	V1	0.6166	0.1445	0.2042	0.7876	0.7835
XGBoost	V2	0.5287	0.1661	0.2264	0.7289	0.7212
XGBoost	V3	0.4494	0.1881	0.2454	0.6724	0.6597
LightGBM	V1	0.6448	0.1379	0.1954	0.8046	0.8033
LightGBM	V2	0.4689	0.1786	0.2395	0.6896	0.6881
LightGBM	V3	0.4284	0.1960	0.2491	0.6554	0.6431
CatBoost	V1	0.6237	0.1371	0.1998	0.7930	0.7925
CatBoost	V2	0.5235	0.1679	0.2259	0.7301	0.7297
CatBoost	V3	0.4284	0.1924	0.2485	0.6619	0.6562
TabPFN	V1	0.5472	0.1584	0.2198	0.7454	0.7409
TabPFN	V2	0.5404	0.1659	0.2233	0.7414	0.7386
TabPFN	V3	0.4467	0.1935	0.2457	0.6751	0.6674

Table 11: GroupShuffleSplit (20 random splits) over held-out broths, v10. Mean across splits. The close agreement with Table 10 shows the estimate is not an artifact of one fold assignment.

Model	Version	R^2	MAE	RMSE
XGBoost	V1	0.5919	0.1499	0.2071
XGBoost	V2	0.4908	0.1771	0.2325
XGBoost	V3	0.4123	0.1990	0.2509
LightGBM	V1	0.5913	0.1501	0.2075
LightGBM	V2	0.4790	0.1810	0.2355
LightGBM	V3	0.3646	0.2115	0.2608
CatBoost	V1	0.5828	0.1510	0.2090
CatBoost	V2	0.4878	0.1795	0.2333
CatBoost	V3	0.3829	0.2058	0.2570

The full model reaches $R^2 \approx 0.62$–0.64 for the trees on broths it has never seen, and even the pure-nutrient model reaches $R^2 \approx 0.43$–0.45. The generalisation gap (interpolation minus group score) is now moderate (≈ 0.21–0.29). It is a sign that the nutrient-to-suppression map is a real function, and the models transfer it to new nutrient environments.

TabPFN shows the same qualitative jump but with a flatter profile across the three feature sets. Its held-out R^2 is 0.547 / 0.540 / 0.447 for V1 / V2 / V3. On the full model this is below the trees (0.55 vs. 0.62–0.64), the same weakness it showed at D=1 and small communities, where the trees performed better, but on the reduced sets it is level with or above them: ***its V2 held-out R^2 (0.540) beats every tree (XGBoost 0.529, CatBoost 0.524, LightGBM 0.469), and its V3 (0.447) is essentially tied with the best tree (XGBoost 0.449) and above the others***. The pattern is consistent with the interpolation table: TabPFN loses a little when there is rich structure for a tree to exploit, and gains when the problem is reduced to nutrients alone.

6.1 Reading the Pearson and Spearman numbers

A very important reading in this report concerns the Pearson and Spearman columns of Table 10. On held-out broths the correlations sit in the band 0.66–0.80: about 0.79–0.80 for the full tree models and about 0.65–0.67 for the pure-nutrient model across all three tree families. TabPFN lands in the same band (0.745 (V1), 0.741 (V2), 0.675 (V3)) on Pearson, with Spearman a bit below each, so the conclusion does not depend on the choice of learner: four independently built models, one of them a pre-trained transformer that never trained on our data at all, all agree that ***unseen nutrient environments can be ranked with correlation in the mid-***0.6***s to high-***0.7***s***.

Pearson r measures whether the predicted suppression moves up and down *in step* with the true suppression; Spearman r measures whether the model *orders* nutrient environments correctly from least to most suppressing, ignoring the exact magnitudes. A value near zero would mean the model's predictions on unseen broths are essentially unrelated to the truth. A value of 0.66 on the pure-nutrient model means that, on broths the model has never seen, its predictions still track and rank the true suppression strongly and reliably. The ordering it produces over new nutrient environments is largely correct.

This is important as good Spearman and Pearson coefficients mean that the model is atleast successful in learning the trends in nutrient to suppression mapping without memorisation. Not to a precision that we would prefer of course, since the error is around 0.1-0.2, but still, relevant nutrient to behaviour mappings are being identified.

The long-term objective is to prescribe a nutrient environment that maximises suppression. For prescription, ranking is what counts: we need the model to tell us which candidate nutrient conditions are more suppressing than others so that an optimiser can climb toward the best ones. A held-out-broth Spearman of 0.65–0.80 says the models already provide a usable ranking signal over unseen nutrient environments, even before any further improvement to absolute accuracy.

7 Attribution: SHAP and Feature-Importance Patterns

The attribution analysis asks ***whether independently trained models converge on the same nutrients and species***. They do, which is strong evidence that the learned signal is a property of the biology in the simulation rather than of any one model.

Three patterns stand out. First, nutrient c8 is the single most important feature in all three models, whether expressed as its concentration `R0_c8` or its presence `active_c8`; c6, c9, c20, and c1 recur as the next tier. Second, ***same species reappear across independently trained models***. For example, `Species_0064` is the top-ranked species in all three, with `Species_0076` and `Species_0080` close behind. Because these strains were never passed any explicit trait features (only a presence/absence bit), their repeated emergence means the models are recovering the same underlying metabolic "genome" signal from membership alone, and agreeing on it.

Table 12: Top SHAP features (full model, V1) for each tree family, in descending order of mean |SHAP|. The same nutrient (c8) leads everywhere, the same secondary nutrients recur, and the same species reappear.

Rank	XGBoost	LightGBM	CatBoost
1	R0_c8	R0_c8	active_c8
2	R0_c6	active_c8	community_size
3	community_size	community_size	R0_c6
4	R0_c9	Species_0064	Species_0064
5	Species_0064	R0_c6	R0_c9
6	R0_c16	R0_c20	active_c6
7	R0_c20	Species_0076	R0_c20
8	R0_c1	R0_c1	R0_c1
9	R0_c17	R0_c17	R0_c8
10	Species_0076	active_c6	R0_c14

8 Further Steps: Optimisation for Nutrient Prescription

We now have a pipeline that ranks unseen nutrient environments by suppression. Now, instead of predicting suppression for a given broth, we have to find the broth (the R_0 vector) that maximises suppression. This is an optimisation problem over the nutrient space, and the trained models serve as the objective.

9 References

1. Marsland, R., Cui, W., Mehta, P. (2020). A minimal model for microbial biodiversity can reproduce experimentally observed ecological patterns. Scientific Reports, 10(1), 3308. https://doi.org/10.1038/s41598-020-60130-2

2. Hollmann, N., Müller, S., Purucker, L., Krishnakumar, A., Körfer, M., Hoo, S. B., Schirrmeister, R. T., Hutter, F. (2025). Accurate predictions on small data with a tabular foundation model. Nature, 637(8045), 319–326. https://doi.org/10.1038/s41586-024-08328-6